

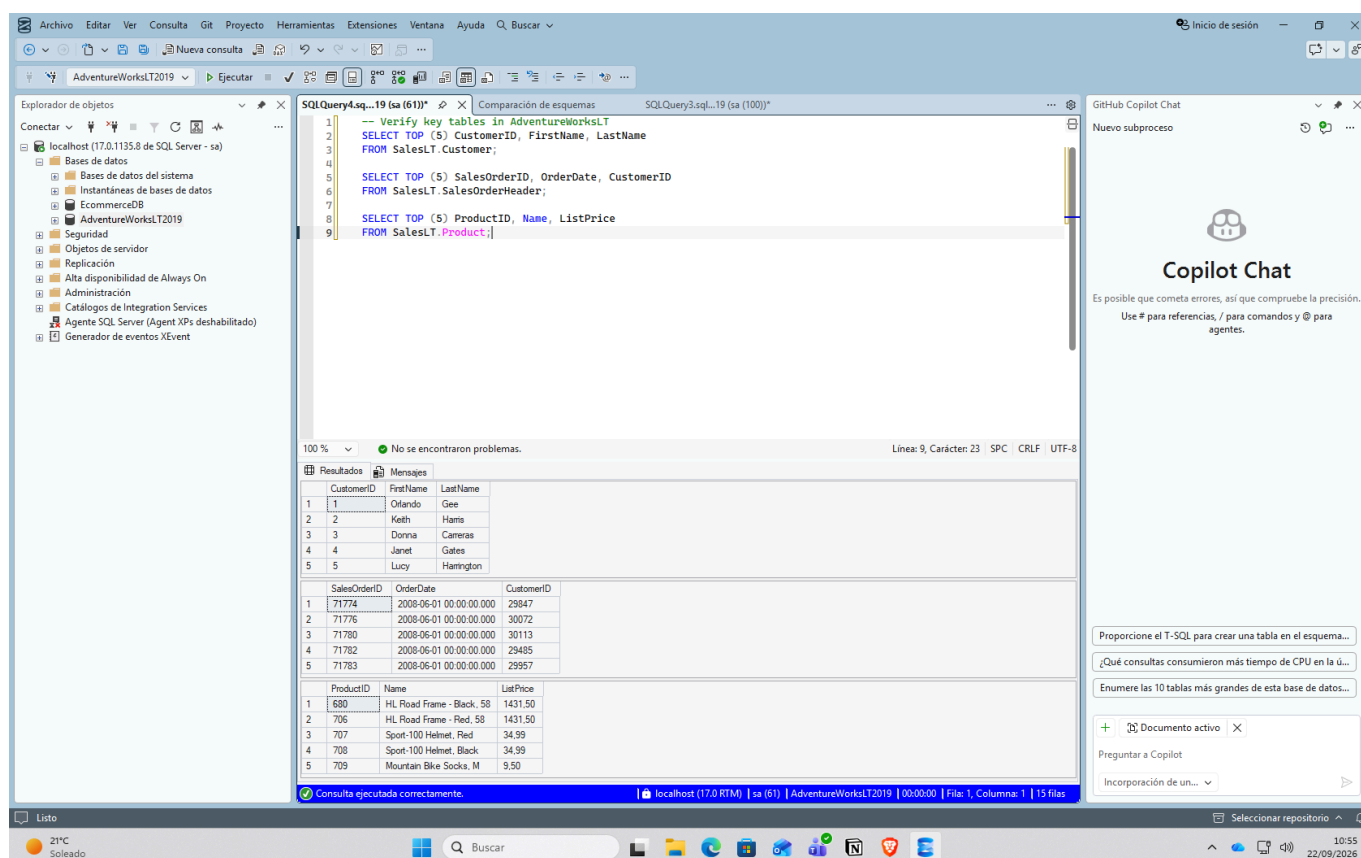
Paso 1. Verificar Base de Datos

En este paso haremos una consulta para verificar si la base de datos se ha restaurado correctamente.

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;

SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;

SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'AdventureWorksLT2019' database. The central pane shows three SQL queries being executed. The bottom pane displays the results of these queries in a grid format.

Query 1: Verify key tables in AdventureWorksLT

```
SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;
```

CustomerID	FirstName	LastName
1	Orlando	Gee
2	Keith	Harris
3	Danna	Cameras
4	Janet	Gates
5	Lucy	Harrington

Query 2: Verify key tables in AdventureWorksLT

```
SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;
```

SalesOrderID	OrderDate	CustomerID
71774	2008-06-01 00:00:00.000	29847
71776	2008-06-01 00:00:00.000	30072
71780	2008-06-01 00:00:00.000	30113
71782	2008-06-01 00:00:00.000	29485
71783	2008-06-01 00:00:00.000	29957

Query 3: Verify key tables in AdventureWorksLT

```
SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```

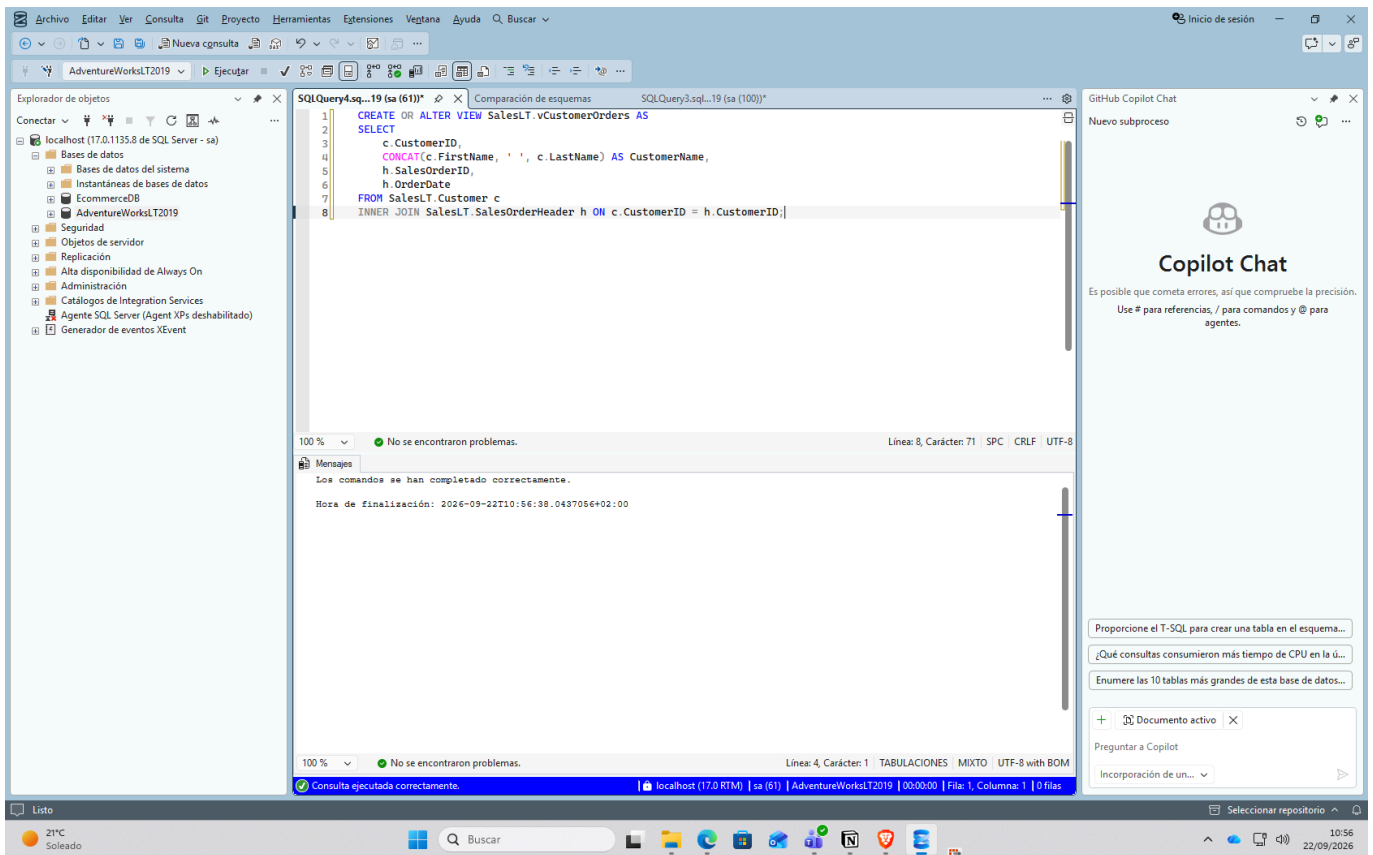
ProductID	Name	ListPrice
680	HL Road Frame - Black, 58	1431.50
706	HL Road Frame - Red, 58	1431.50
707	Sport-100 Helmet, Red	34.99
708	Sport-100 Helmet, Black	34.99
709	Mountain Bike Socks, M	9.50

Paso 2. Crear una view para simplificar consultas

En este paso creamos una view que combina los clientes con sus pedidos dentro del schema SalesLT.

```
CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
```

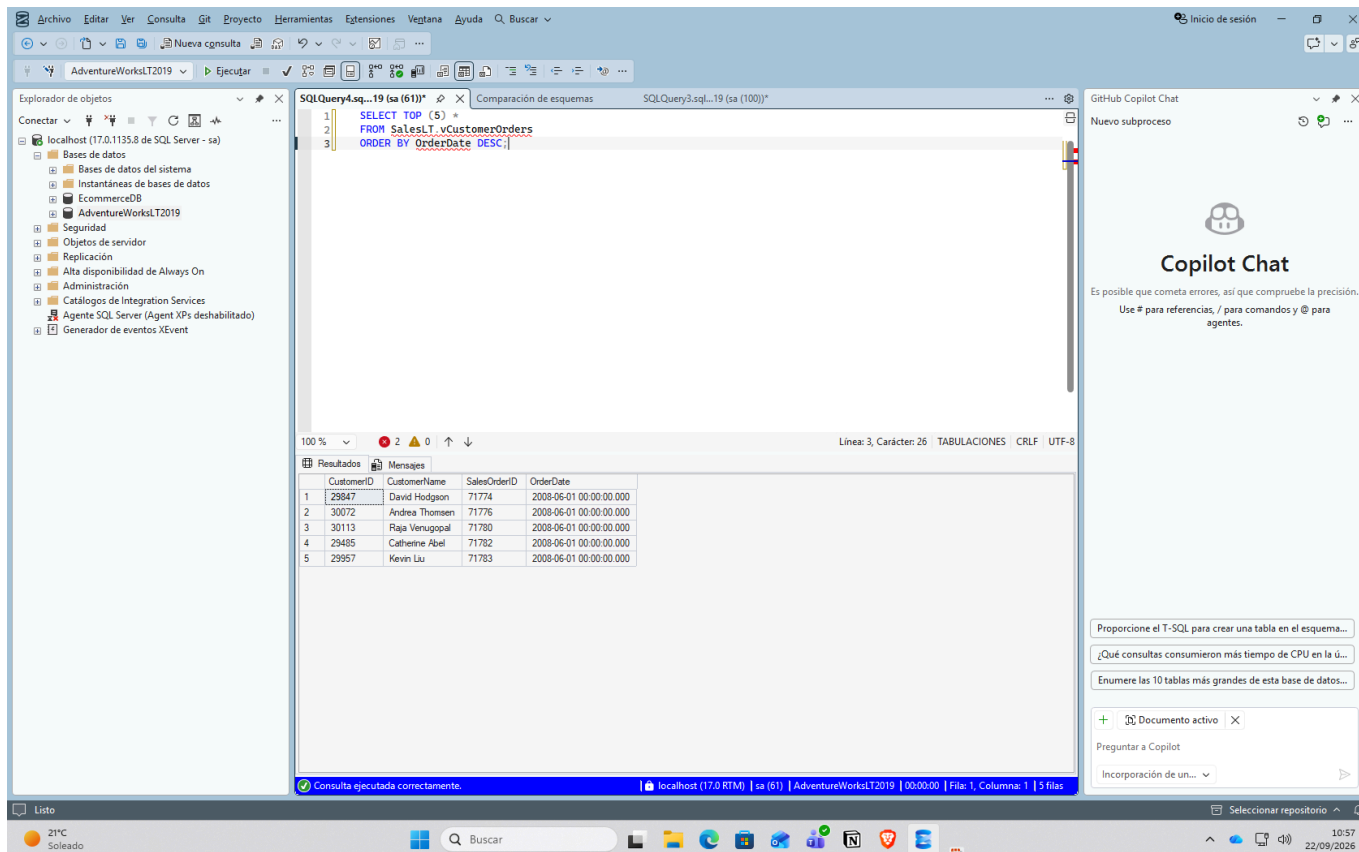
```
h.OrderDate
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```



Paso 3. Validamos que la view es correcta

En este paso comprobamos que la view devuelve hasta cinco filas con los pedidos más recientes.

```
SELECT TOP (5) *
FROM SalesLT.vCustomerOrders
ORDER BY OrderDate DESC;
```



Paso 4. Crear un stored procedure para procesar un pedido

En este paso creamos un store procedure

```
CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
    @SalesOrderID INT,
    @ProductID INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    -- Use Product ListPrice as UnitPrice
    DECLARE @UnitPrice DECIMAL(18,2);
    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
    FROM SalesLT.Product
    WHERE ProductID = @ProductID;

    IF @UnitPrice IS NULL
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Invalid ProductID specified.', 1;
    END

    -- Ensure SalesOrderID exists
```

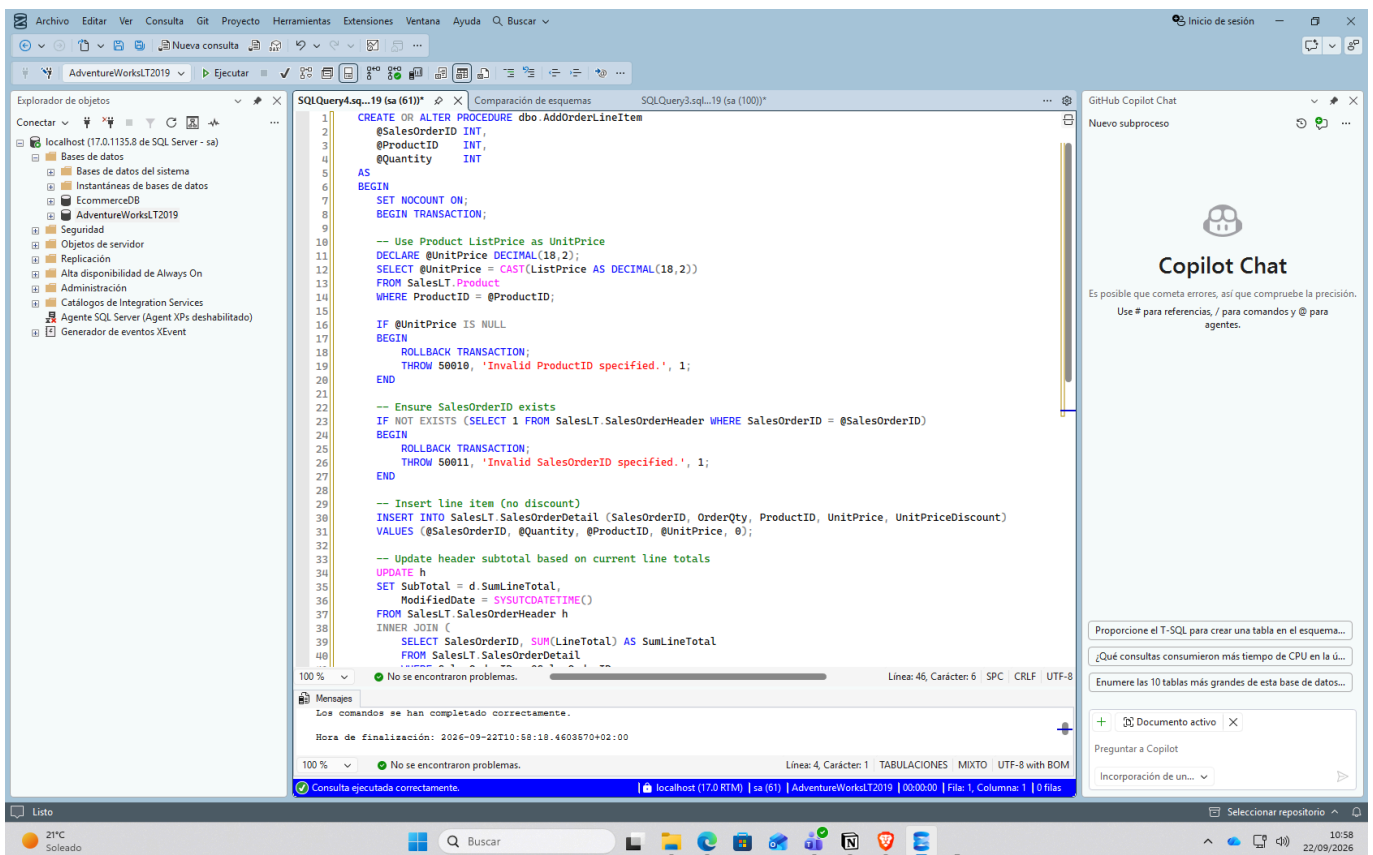
```
IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID =
@SalesOrderID)
BEGIN
    ROLLBACK TRANSACTION;
    THROW 50011, 'Invalid SalesOrderID specified.', 1;
END

-- Insert line item (no discount)
INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID,
UnitPrice, UnitPriceDiscount)
VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

-- Update header subtotal based on current line totals
UPDATE h
SET SubTotal = d.SumLineTotal,
    ModifiedDate = SYSUTCDATETIME()
FROM SalesLT.SalesOrderHeader h
INNER JOIN (
    SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @SalesOrderID
    GROUP BY SalesOrderID
) d ON d.SalesOrderID = h.SalesOrderID;

COMMIT TRANSACTION;

END;
```



Paso 5. Probamos que el stored procedure se ha creado correctamente

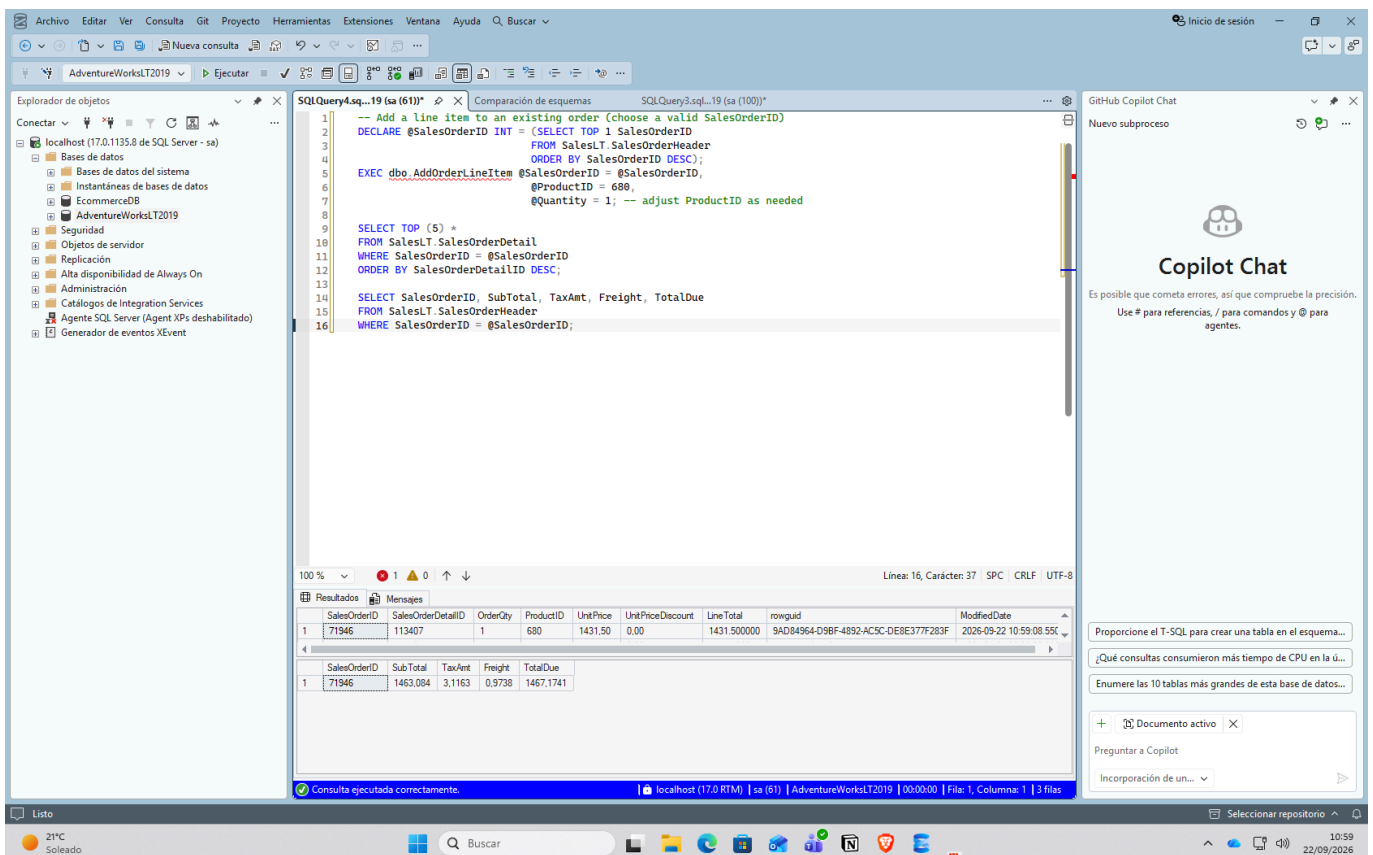
En este paso realizamos una consulta para comprobar el stored procedure

```
-- Add a line item to an existing order (choose a valid SalesOrderID)
DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
                              FROM SalesLT.SalesOrderHeader
                              ORDER BY SalesOrderID DESC);

EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
                          @ProductID = 680,
                          @Quantity = 1; -- adjust ProductID as needed

SELECT TOP (5) *
FROM SalesLT.SalesOrderDetail
WHERE SalesOrderID = @SalesOrderID
ORDER BY SalesOrderDetailID DESC;

SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
FROM SalesLT.SalesOrderHeader
WHERE SalesOrderID = @SalesOrderID;
```



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'AdventureWorksLT2019' database. The central pane shows the execution of a stored procedure and a query. The right pane displays the 'Copilot Chat' interface.

SQL Query Results:

SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	Line Total	rowguid	ModifiedDate
71946	113407	1	680	1431.50	0.00	1431.500000	9AD84964-D9BF-4892-AC5C-DEBE377F283F	2026-09-22 10:59:08.550

SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
71946	1463.084	3.1163	0.9738	1467.1741

Copilot Chat Interface:

GitHub Copilot Chat

Nuevo subproceso

Copilot Chat

Es posible que cometa errores, así que compruebe la precisión. Use # para referencias, / para comandos y @ para agentes.

Proporcione el T-SQL para crear una tabla en el esquema...

¿Qué consultas consumieron más tiempo de CPU en la ú...

Enumere las 10 tablas más grandes de esta base de datos...

+ Documento activo X

Preguntar a Copilot

Incorporación de un...

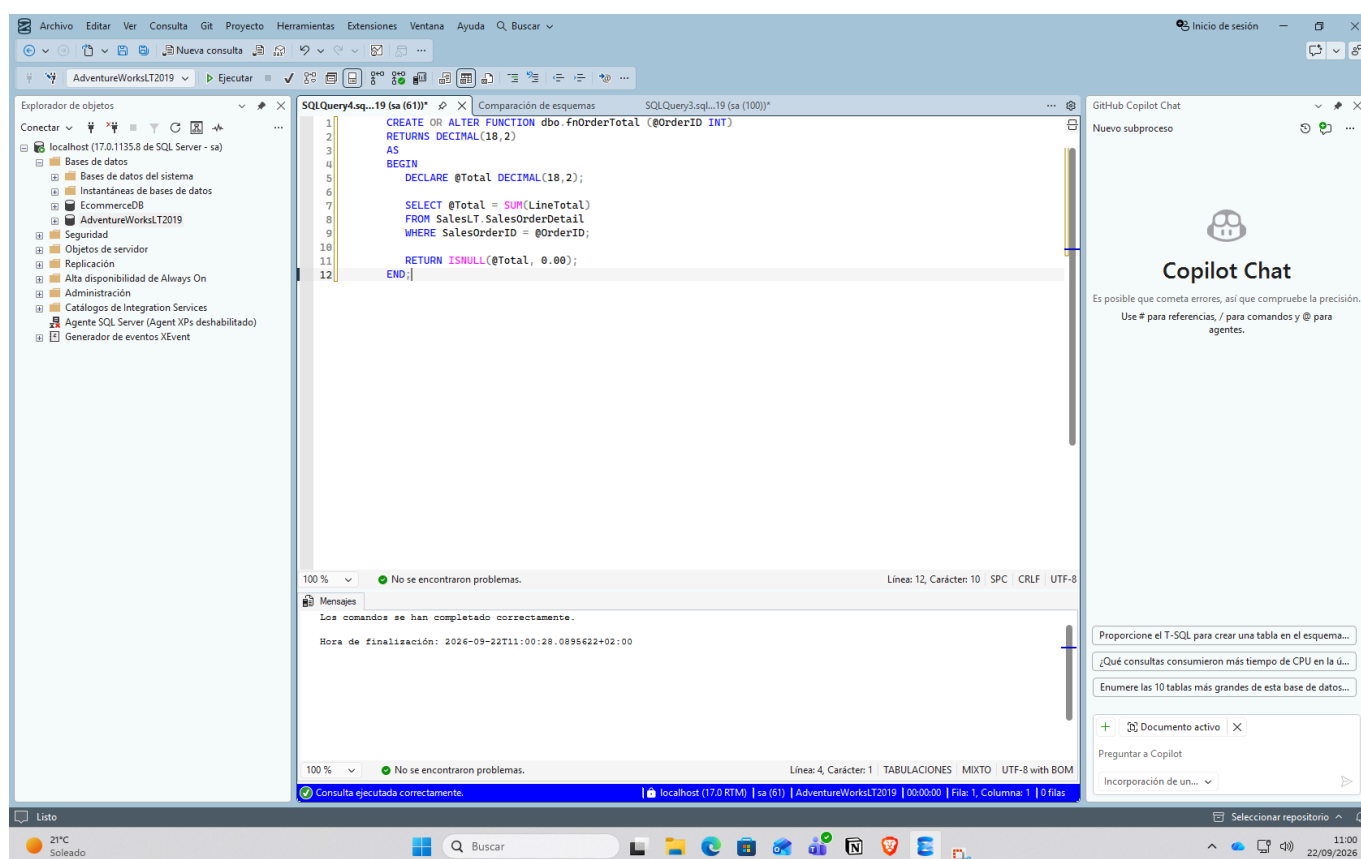
Paso 6. Crear una scalar function para cálculos reutilizables

En este paso crearemos un scalar function que devolverá el valor total de un pedido utilizando los LineTotal de AdventureWorksLT.

```
CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @Total DECIMAL(18,2);

    SELECT @Total = SUM(LineTotal)
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @OrderID;

    RETURN ISNULL(@Total, 0.00);
END;
```



Paso 7. Utilizar scalar function

En este paso utilizaremos el scalar function que hemos creado.

```
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```

The screenshot displays the SQL Server Enterprise Edition interface. The central pane shows the execution results of a query. The query is as follows:

```
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```

The results table has two columns: SalesOrderID and OrderTotal. The data is sorted in descending order of OrderTotal.

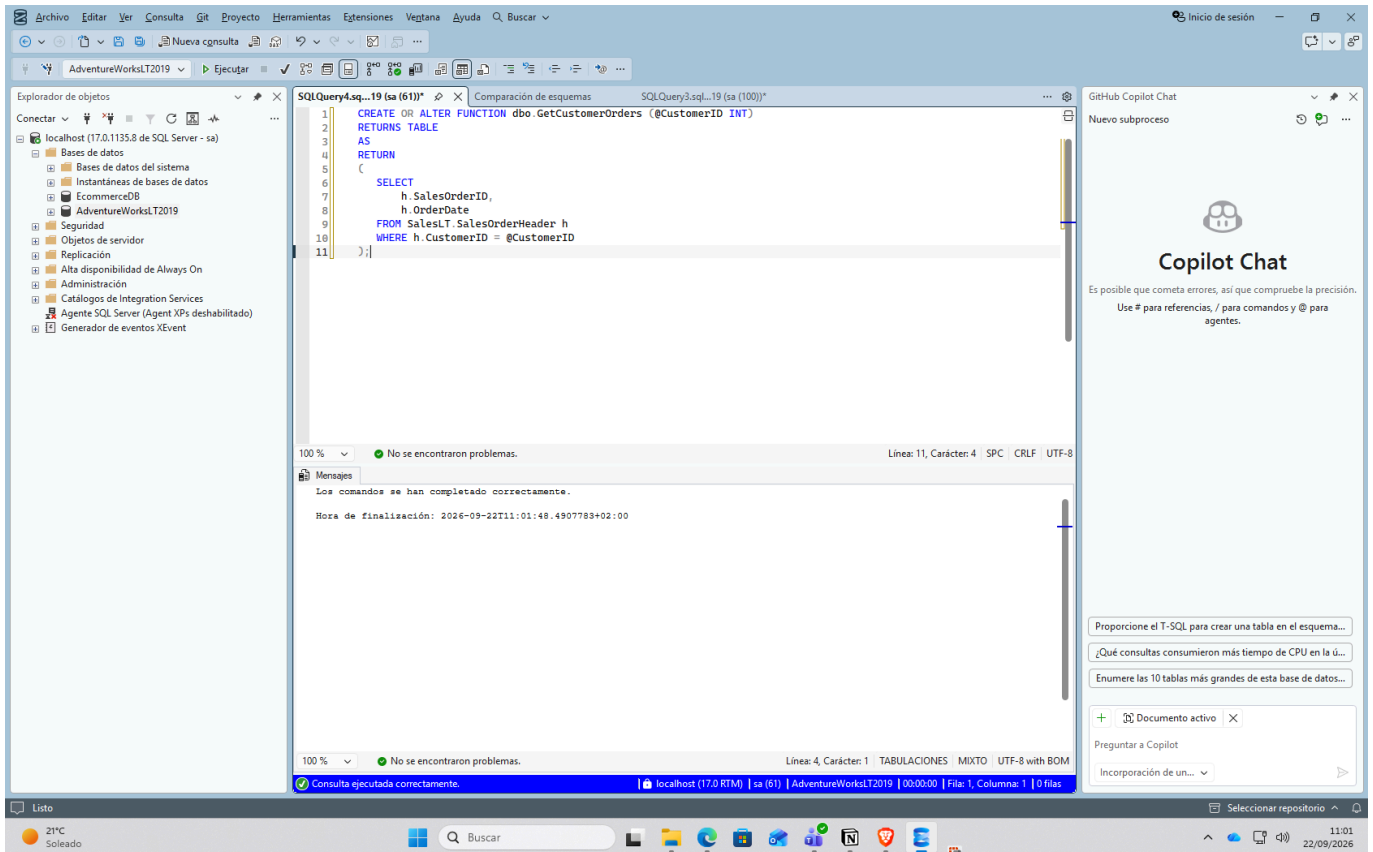
SalesOrderID	OrderTotal
71946	1463.08
71938	74160.23
71936	79589.62
71935	5533.87
71923	96.11
71920	2527.13
71917	37.76
71915	1732.89
71902	59894.21
71899	1856.21
71898	53248.69
71897	10585.05
71895	221.26
71885	524.66
71867	858.90
71863	2777.14
71858	11528.84
71856	500.30
71846	1884.39
71845	34118.54
71832	28950.68
71831	1712.95
71816	2847.41
71815	926.92
71797	65123.46
71796	47848.03
71784	89869.28
71783	65683.37
71782	33319.99
71780	29821.01

On the right side, the GitHub Copilot Chat interface is visible, showing a chat window with the Copilot logo and a message: "Es posible que cometa errores, así que compruebe la precisión. Use # para referencias, / para comandos y @ para agentes."

Paso 8. Crear una inline table-valued function (TVF)

En este paso creamos una inline table-valued function que devolverá los pedidos de un determinado cliente.

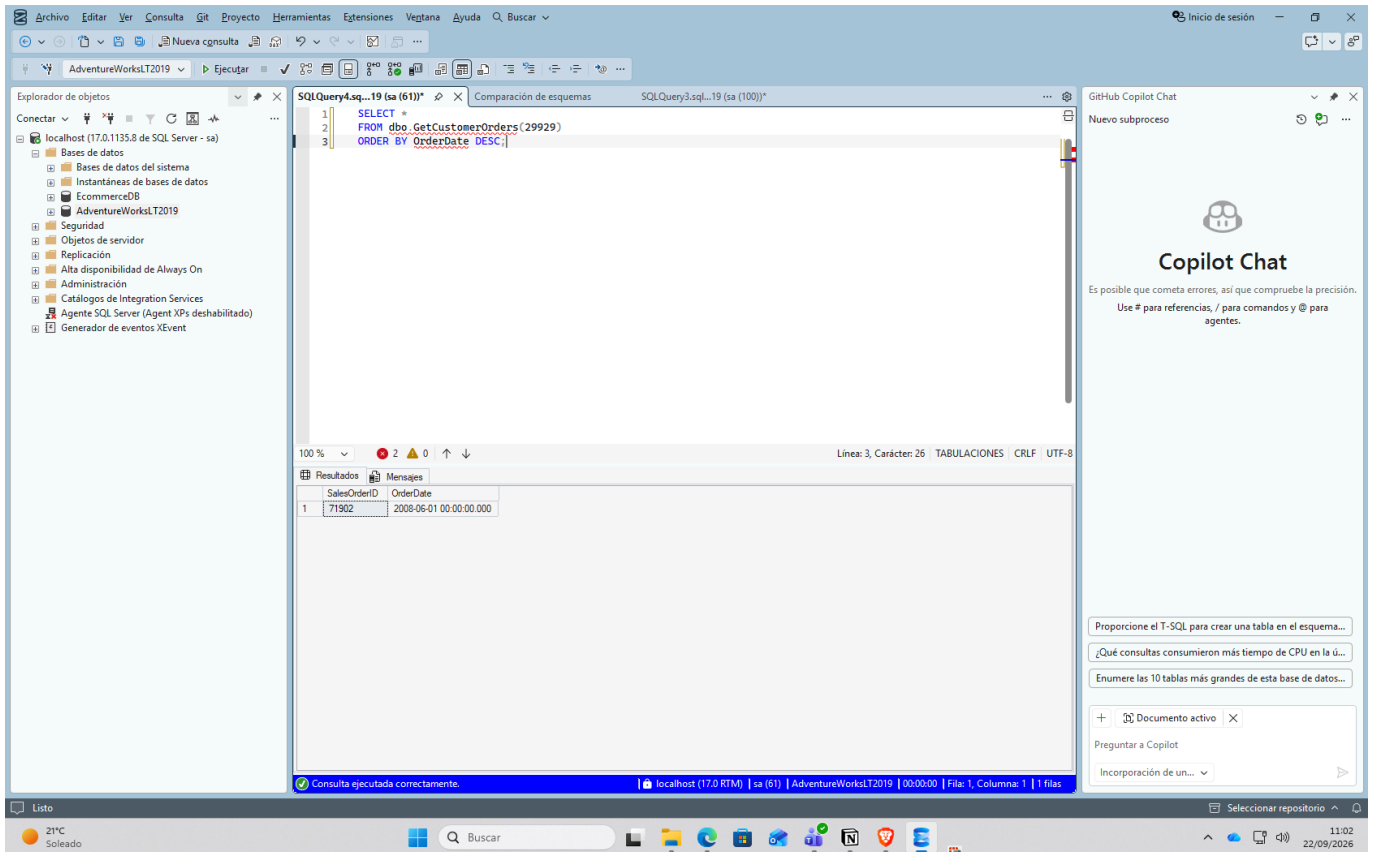
```
CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        h.SalesOrderID,
        h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);
```



Paso 9. Consultar TVF

En este paso consultamos la table-valued function

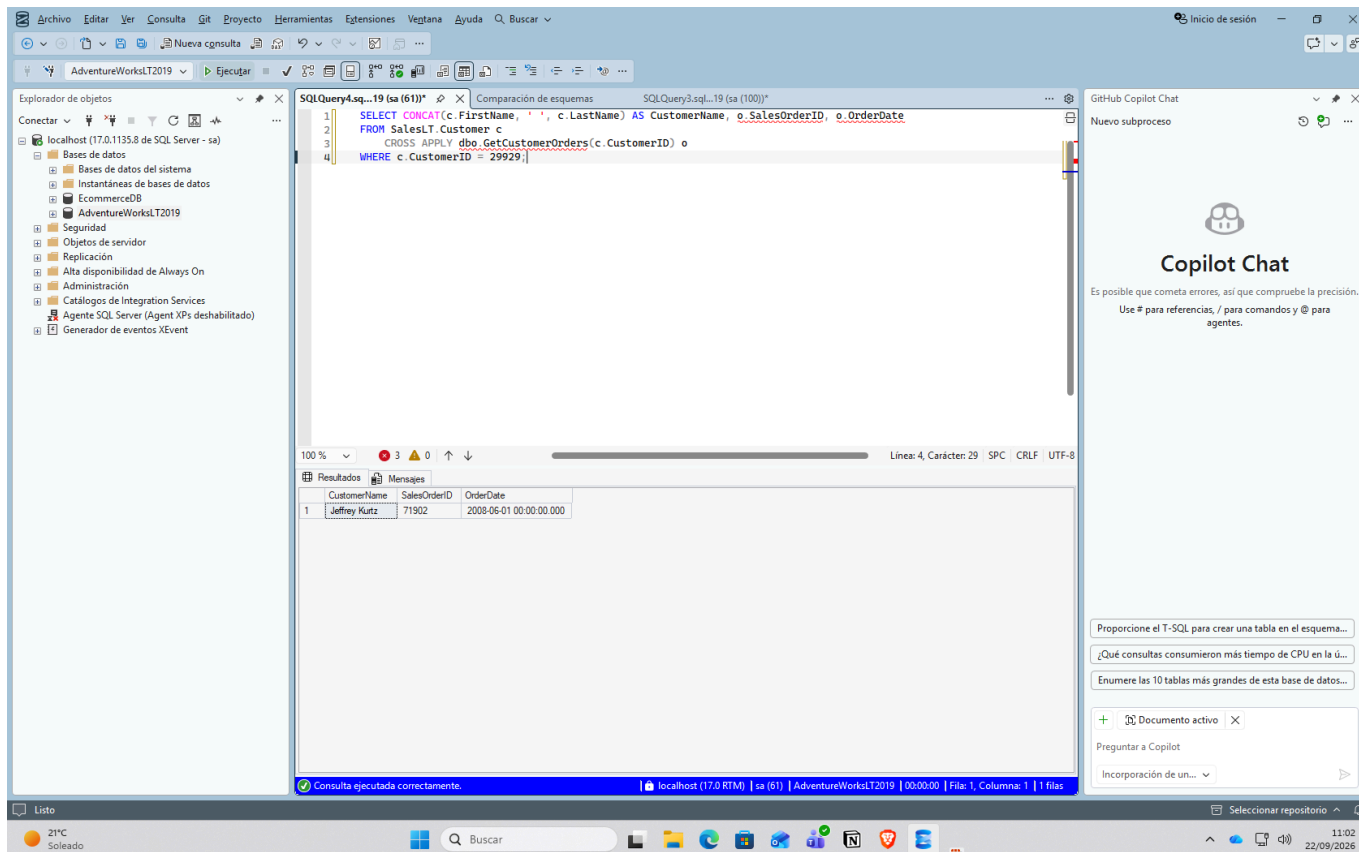
```
SELECT *
FROM dbo.GetCustomerOrders(29929)
ORDER BY OrderDate DESC;
```

Paso 10. Combinar TVF con clientes

En este paso combinaremos table-valued function con los clientes.

```
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID,
o.OrderDate
FROM SalesLT.Customer c
CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE c.CustomerID = 29929;
```



Paso 11. Crear un trigger para registrar cambios

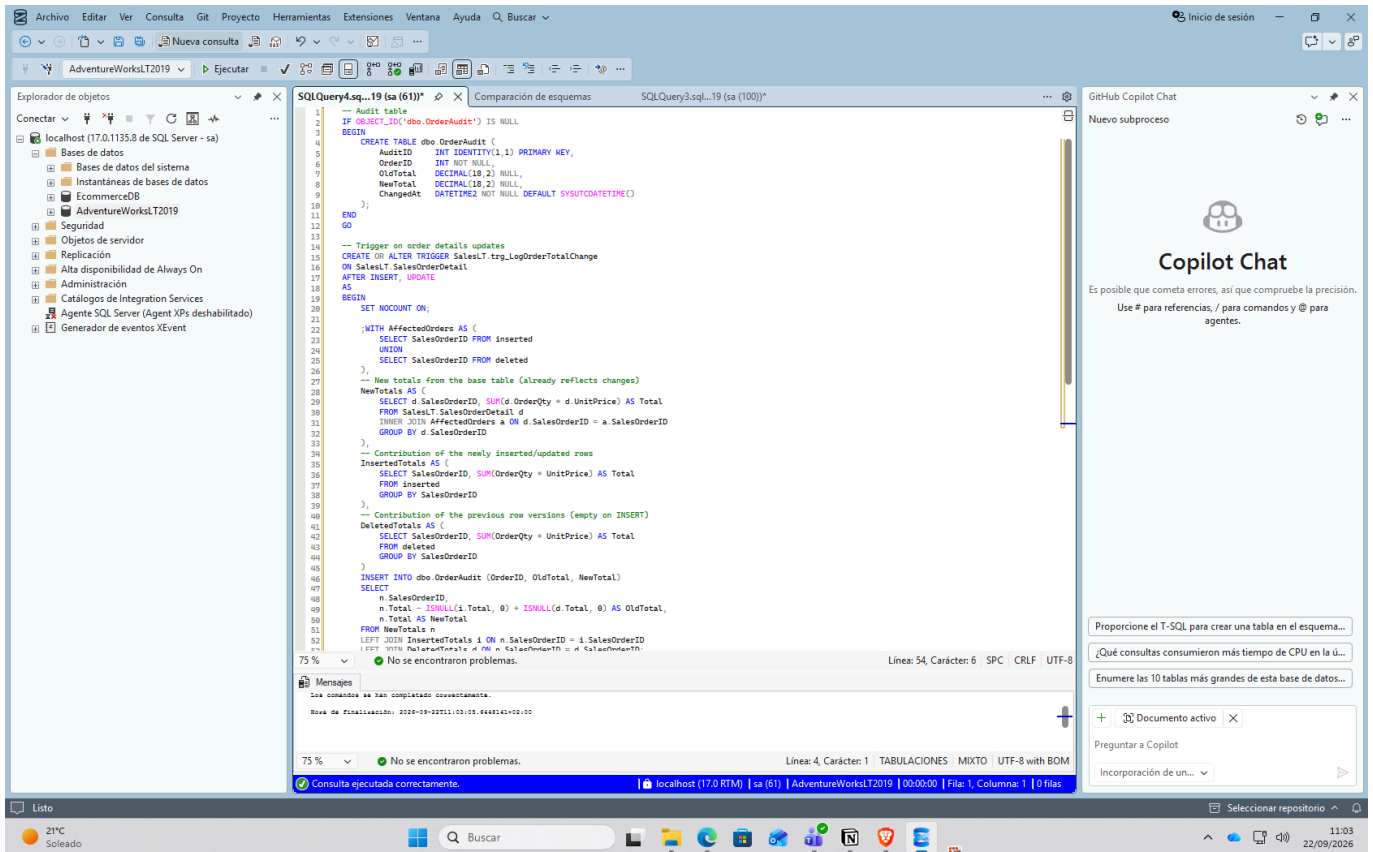
En este paso añadiremos un trigger que registre cambios en los totales de los pedidos cuando cambien los detalles de los pedidos de SalesLT.

```
-- Audit table
IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID      INT IDENTITY(1,1) PRIMARY KEY,
        OrderID      INT NOT NULL,
        OldTotal     DECIMAL(18,2) NULL,
        NewTotal     DECIMAL(18,2) NULL,
        ChangedAt    DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
END
GO

-- Trigger on order details updates
CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
```

```
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    -- New totals from the base table (already reflects changes)
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    -- Contribution of the newly inserted/updated rows
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted
        GROUP BY SalesOrderID
    ),
    -- Contribution of the previous row versions (empty on INSERT)
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted
        GROUP BY SalesOrderID
    )
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
    SELECT
        n.SalesOrderID,
        n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
        n.Total AS NewTotal
    FROM NewTotals n
    LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
    LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;
```



Paso 12. Comprobar que funciona el trigger

En este paso se comprobará si el trigger funciona.

```
-- Update an order detail to change the total
UPDATE d
SET OrderQty = OrderQty + 1
FROM SalesLT.SalesOrderDetail d
WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader
ORDER BY SalesOrderID DESC);

SELECT TOP (5) *
FROM dbo.OrderAudit
ORDER BY AuditID DESC;
```

